CampusCompute

OS Mini-Project (UE25CS242A), Level 3

1. Team

Name	Full SRN	Code ownership
A B Sharvesh	PES1UG25CS001	main.c, campus.h, job.c, sched.c, banker.c, scheduling and Banker's test files
Abhinav Akinepalli	PES1UG25CS013	logger.c, ipc.c, worker.c, rlimit.c, integrate.c, sysutil.c, README

Description:

CampusCompute is a command-line program in C that simulates a shared university computing lab and demonstrates key operating-system concepts. Each feature is a separate command, so it can be run and tested on its own.

For scheduling, jobs are read from a file and validated, then run under FCFS, Round Robin and non-preemptive Priority scheduling. Ties are broken by earlier arrival, then smaller job ID. The program prints each execution order with completion, turnaround and waiting times and their averages, and a comparison mode runs all three policies on the same jobs. The Banker's algorithm computes Need as Maximum minus Allocation, then finds a safe sequence for a safe state or reports an unsafe one.

The logger uses producer and consumer threads sharing a bounded queue, protected by a mutex and condition variables, and writes events to a log file. The threads shut down in an orderly way, so no event is lost. For inter-process communication, a writer and a forked reader exchange messages through POSIX shared memory, coordinated by two named semaphores. Worker processes are created with fork() and collected with waitpid(). A clearly labeled simulated failure shows the parent detecting a worker that exits abnormally. The resource-limit demo uses setrlimit() in a child process to limit open files, memory and CPU time, and shows the effect of each.

Every demo cleans up its threads, child processes, shared memory and semaphores. One command runs all the features and prints a PASS/FAIL summary, which also confirms that nothing was left behind. Invalid commands and bad input produce clear error and usage messages.

S01 – Environment and SRN prompt

```
[PES1UG25CS013] ~/campuscomute$ echo $SRN; pwd; uname -a; lsb_release -d
PES1UG25CS013
/home/abhinav-a/campuscomute
Linux abhinav-a-VirtualBox 7.0.0-38-generic #38-Ubuntu SMP PREEMPT_DYNAMIC Fri Sep  4 09:10:14 UTC 2026 x86_64 GNU/Linux
Description:    Ubuntu 26.04 LTS
```

```
[PES1UG25CS001] ~/campuscompute$ echo $SRN; pwd; uname -a
PES1UG25CS001
/home/berkman/campuscompute
Linux berkman-VirtualBox 7.0.0-38-generic #38-Ubuntu SMP PREEMPT_DYNAMIC Fri Sep  4 09:10:14 UTC 2026 x86_64 GNU/Linux
[PES1UG25CS001] ~/campuscompute$
```

S01: Ubuntu/WSL terminal showing my full SRN in the prompt and the project working directory.

S02 – Compiler and build tools

```
[PES1UG25CS013] ~/campuscomute$ gcc --version; make --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

GNU Make 4.4.1
Built for x86_64-pc-linux-gnu
Copyright (C) 1988-2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

```
[PES1UG25CS001] ~/campuscompute$ gcc --version; make --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

GNU Make 4.4.1
Built for x86_64-pc-linux-gnu
Copyright (C) 1988-2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
[PES1UG25CS001] ~/campuscompute$
```

S02: GCC and Make are installed; their version output is shown.

S03 – Project files

```
[PES1UG25CS013] ~/campuscomute$ ls -R
.:
campuscompute_partB

./campuscompute_partB:
Makefile  README_partB.md  include  src  tests

./campuscompute_partB/include:
campus.h  sysutil.h

./campuscompute_partB/src:
integrate.c  ipc.c  logger.c  main.c  rlimit.c  stubs_a.c  sysutil.c  worker.c

./campuscompute_partB/tests:
banker_safe.txt  banker_unsafe.txt  jobs.txt
```

```
[PES1UG25CS001] ~/campuscompute$ ls -R
.:
Makefile  campus  include  src  tests

./include:
campus.h  job.h

./src:
banker.c  job.c  main.c  sched.c  stubs_b.c

./tests:
banker_bad.txt  banker_safe.txt  banker_unsafe.txt  banker_unsafe_partial.txt  jobs.txt  jobs_bad.txt  jobs_gap_ties.txt  jobs_ties5.txt  run_tests.sh
[PES1UG25CS001] ~/campuscompute$
```

S03: Project directory listing with the source files and the Makefile.

S04 – Clean build

```
[PES1UG25CS013] ~/campuscompute$ make clean && make
rm -f campus logs/*.log
gcc -Wall -Wextra -g -Iinclude src/integrate.c src/ipc.c src/logger.c src/main.c src/rlimit.c src/stubs_a.c src/sysutil.c src/worker.c -o campus -pthread -lrt
[PES1UG25CS013] ~/campuscompute$
```

```
[PES1UG25CS001] ~/campuscompute$ make clean && make
rm -f campus logs/*.log
gcc -Wall -Wextra -g -Iinclude src/banker.c src/job.c src/main.c src/sched.c src/stubs_b.c -o campus -pthread -lrt
[PES1UG25CS001] ~/campuscompute$
```

S04: Clean build from source with no compiler errors.

S05 – CLI help / usage

```
[PES1UG25CS001] ~/campuscompute$ ./campus help
CampusCompute - job scheduling and resource management prototype
usage: campus <command> [arguments]
  sched fcfs|rr|priority|compare <jobfile> [-q N]   CPU scheduling (default quantum 2)
  banker <file>                                     Banker's safety algorithm
  logger [producers] [consumers] [items]            producer-consumer logger (threads)
  ipc                                               shared memory + named semaphores
  worker normal|fail|signal                         fork/waitpid, simulated failures
  rlimit [nofile|mem|cpu|all]                       setrlimit demonstrations
  all                                               run every feature, print summary
  help                                              show this message
Examples:
  ./campus sched rr tests/jobs.txt -q 3
  ./campus banker tests/banker_safe.txt
Job file : one job per line "id arrival burst priority"; '#' starts a comment;
           smaller priority number = higher priority.
Rules    : tie-break = earlier arrival, then smaller job id.
           Round Robin: a job arriving exactly when a quantum expires is queued before the pre-empted job.
Exit code: 0 = success; banker: 0 safe, 1 unsafe; 2 = usage or bad input.
[PES1UG25CS001] ~/campuscompute$
```

S05: The help command lists every available command and its usage.

S06 – FCFS scheduling

```
berkman@berkman-VirtualBox:~/campusco
                                                    ~/campuscompute

[PES1UG25CS001] ~/campuscompute$ ./campus sched fcfs tests/jobs.txt

=== FCFS (First-Come-First-Served) ===
Input file : tests/jobs.txt (4 jobs)
Tie-break  : earlier arrival, then smaller job id

Timeline (Gantt chart):
| J1 | J2 | J3 | J4 |
0    5    8    16   22

Execution order: J1 -> J2 -> J3 -> J4

Job    Arrival  Burst  Prio  Start  Completion  Turnaround  Waiting  Status
J1           0      5     2      0           5           5        0    DONE
J2           1      3     1      5           8           7        4    DONE
J3           2      8     4      8          16          14        6    DONE
J4           3      6     3     16          22          19       13    DONE

Average completion time : 12.75
Average turnaround time : 11.25
Average waiting time    : 5.75
Total time (makespan)   : 22
[PES1UG25CS001] ~/campuscompute$
```

S06: FCFS execution order with completion, turnaround and waiting times and their averages.

S07 – Round Robin scheduling

```
[PES1UG25CS001] ~/campuscompute$ ./campus sched rr tests/jobs.txt -q 2

=== Round Robin (time quantum = 2) ===
Input file : tests/jobs.txt (4 jobs)
Tie-break  : earlier arrival, then smaller job id
Quantum    : 2 time units
Queue rule : a job arriving exactly when a quantum expires is queued before the pre-empted job

Timeline (Gantt chart):
| J1 | J2 | J3 | J1 | J4 | J2 | J3 | J1 | J4 | J3 | J4 | J3 |
0    2    4    6    8    10   11   13   14   16   18   20   22

Execution order: J1 -> J2 -> J3 -> J1 -> J4 -> J2 -> J3 -> J1 -> J4 -> J3 ->
                 J4 -> J3

Job    Arrival  Burst  Prio  Start  Completion  Turnaround  Waiting  Status
J1           0      5     2      0          14          14        9    DONE
J2           1      3     1      2          11          10        7    DONE
J3           2      8     4      4          22          20       12    DONE
J4           3      6     3      8          20          17       11    DONE

Average completion time : 16.75
Average turnaround time : 15.25
Average waiting time    : 9.75
Total time (makespan)   : 22
[PES1UG25CS001] ~/campuscompute$
```

S07: Round Robin with the stated time quantum, execution order and scheduling metrics.

S08 – Priority scheduling

```
berkman@berkman-VirtualBox:~/campuscomp
~/campuscompute

[PES1UG25CS001] ~/campuscompute$ ./campus sched priority tests/jobs.txt

=== Priority (non-preemptive) ===
Input file : tests/jobs.txt (4 jobs)
Tie-break  : earlier arrival, then smaller job id
Priority   : smaller number = higher priority (0 is highest)

Timeline (Gantt chart):
| J1 | J2 | J4 | J3 |
0    5    8    14   22

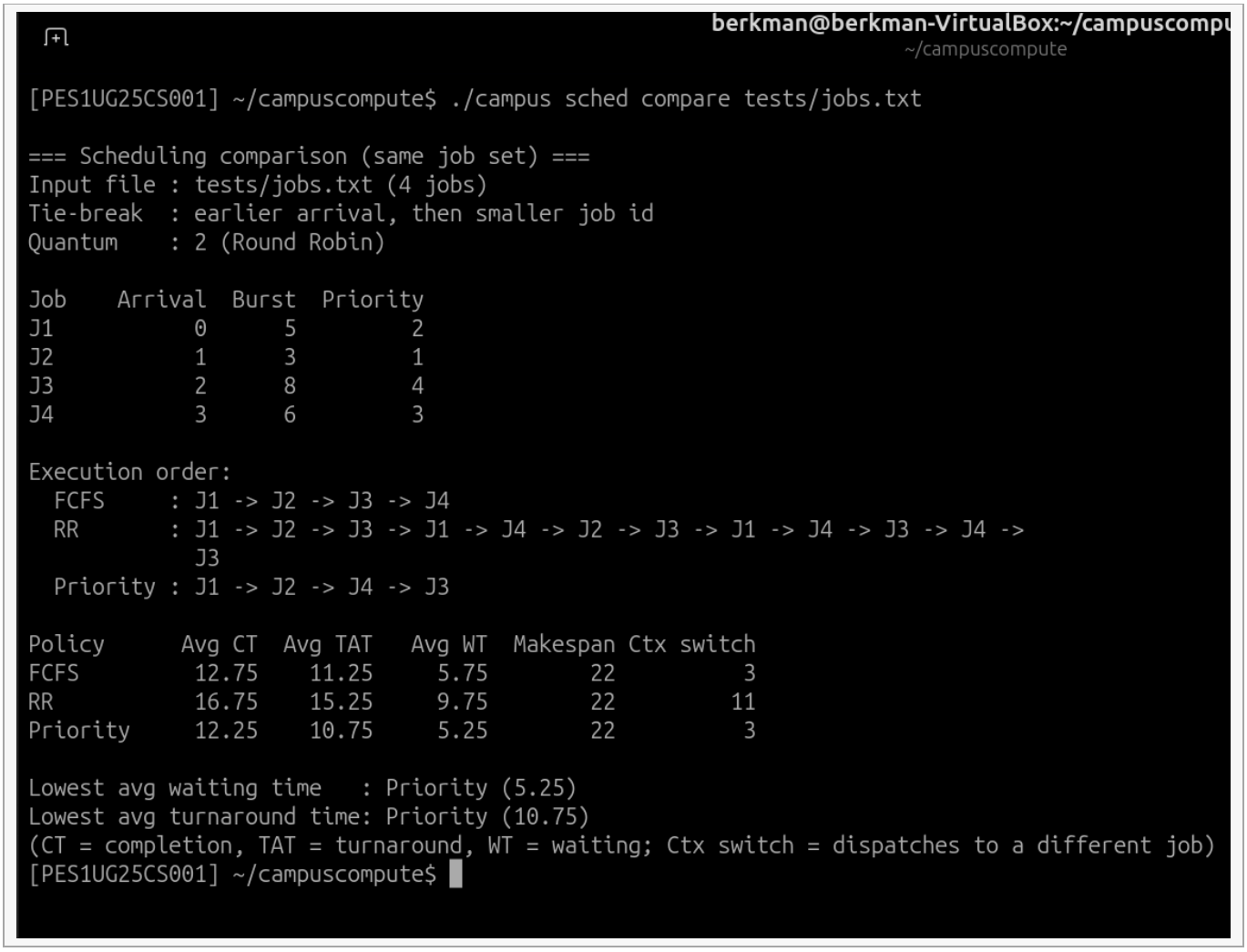
Execution order: J1 -> J2 -> J4 -> J3

Job    Arrival  Burst  Prio  Start  Completion  Turnaround  Waiting  Status
J1           0      5     2      0           5           5        0    DONE
J2           1      3     1      5           8           7        4    DONE
J3           2      8     4     14          22          20       12    DONE
J4           3      6     3      8          14          11        5    DONE

Average completion time : 12.25
Average turnaround time : 10.75
Average waiting time    : 5.25
Total time (makespan)   : 22
[PES1UG25CS001] ~/campuscompute$
```

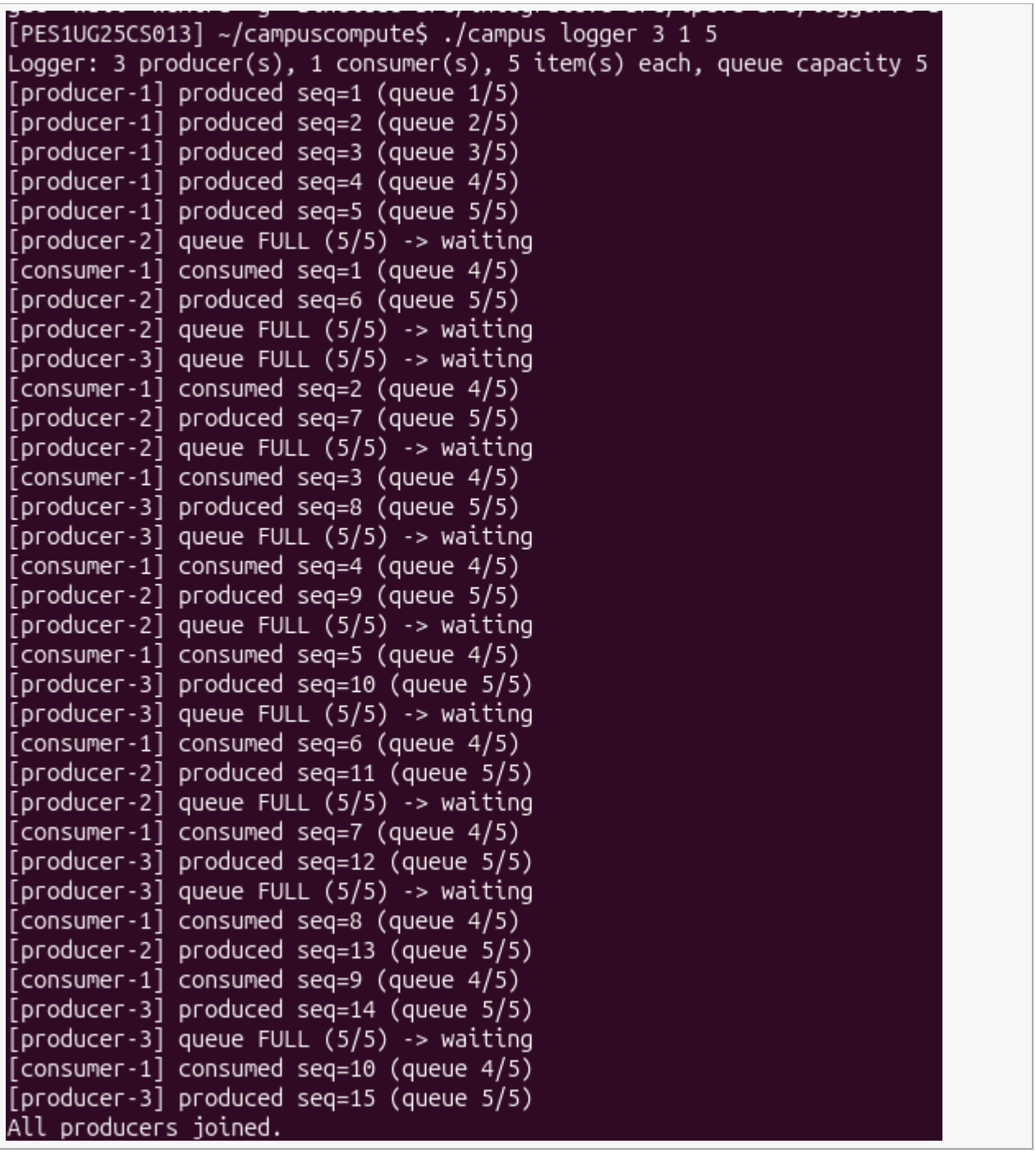
S08: Non-preemptive priority scheduling with job priorities, the tie-break rule and metrics.

S09 – Scheduling comparison



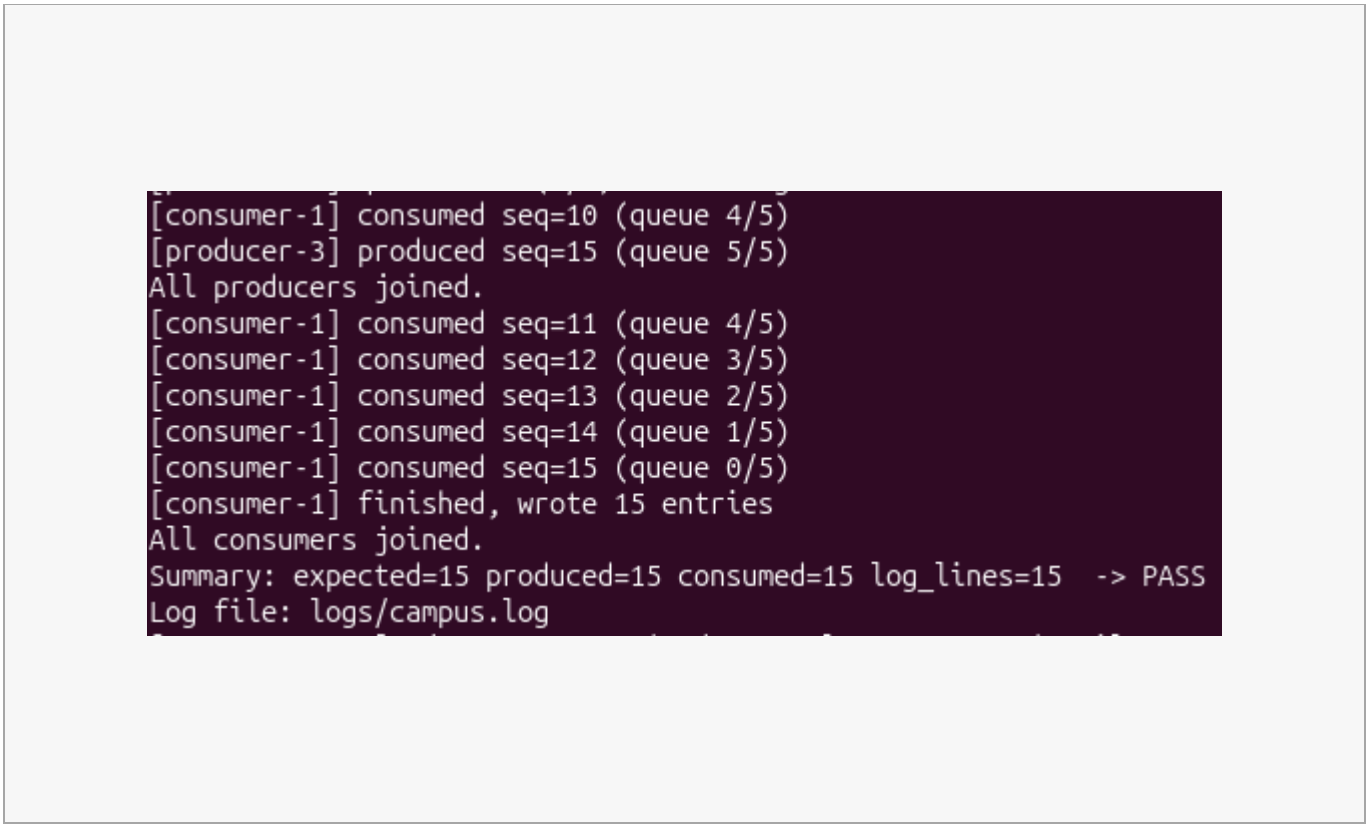
S09: FCFS, Round Robin and Priority compared on the same job set.

S10 – Producer-consumer logger



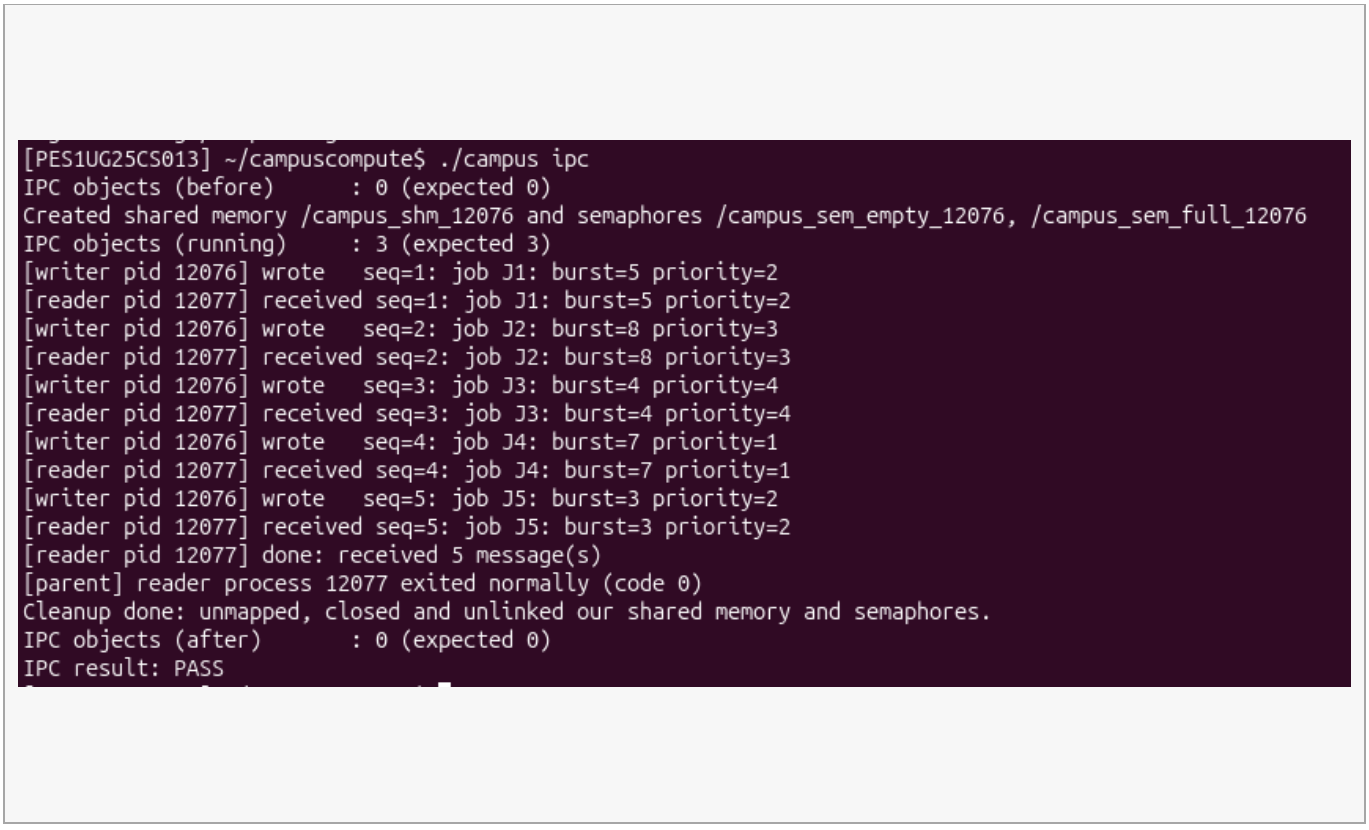
S10: Producers and the consumer exchanging events through the bounded queue, including full and empty waits.

S11 – Thread synchronization and termination



S11: Threads are joined in order and the produced, consumed and logged counts match.

S12 – Shared-memory IPC



S12: Writer and reader processes exchanging messages through POSIX shared memory.

S13 – Semaphore coordination and cleanup

```
[PES1UG25CS013] ~/campuscompute$ ./campus ipc; ls /dev/shm | grep campus
IPC objects (before)      : 0 (expected 0)
Created shared memory /campus_shm_12318 and semaphores /campus_sem_empty_12318, /campus_sem_full_12318
IPC objects (running)     : 3 (expected 3)
[writer pid 12318] wrote   seq=1: job J1: burst=5 priority=2
[reader pid 12319] received seq=1: job J1: burst=5 priority=2
[writer pid 12318] wrote   seq=2: job J2: burst=8 priority=3
[reader pid 12319] received seq=2: job J2: burst=8 priority=3
[writer pid 12318] wrote   seq=3: job J3: burst=4 priority=4
[reader pid 12319] received seq=3: job J3: burst=4 priority=4
[writer pid 12318] wrote   seq=4: job J4: burst=7 priority=1
[reader pid 12319] received seq=4: job J4: burst=7 priority=1
[writer pid 12318] wrote   seq=5: job J5: burst=3 priority=2
[reader pid 12319] received seq=5: job J5: burst=3 priority=2
[reader pid 12319] done: received 5 message(s)
[parent] reader process 12319 exited normally (code 0)
Cleanup done: unmapped, closed and unlinked our shared memory and semaphores.
IPC objects (after)       : 0 (expected 0)
IPC result: PASS
```

S13: Named semaphores coordinate the exchange and every IPC object created is removed afterwards.

S14 – Banker's safe state

```
                                                                    ~/campuscompute

[PES1UG25CS001] ~/campuscompute$ ./campus banker tests/banker_safe.txt
=== Banker's safety check ===
State file : tests/banker_safe.txt
Processes  : 5, resource types: 3
Need = Maximum - Allocation

        Allocation   Maximum      Need
          A  B  C      A  B  C      A  B  C
  P0      0  1  0      7  5  3      7  4  3
  P1      2  0  0      3  2  2      1  2  2
  P2      3  0  2      9  0  2      6  0  0
  P3      2  1  1      2  2  2      0  1  1
  P4      0  0  2      4  3  3      4  3  1

  Available:  3  3  2   (resource types A B C)

Safety algorithm: Work = Available; scan P0..P4 in index order, repeat until no progress.
  Pass 1:
    P0: Need [7 4 3] > Work [3 3 2]  -> not enough, skip
    P1: Need [1 2 2] <= Work [3 3 2] -> can finish, releases its Allocation, Work = [5 3 2]
    P2: Need [6 0 0] > Work [5 3 2]  -> not enough, skip
    P3: Need [0 1 1] <= Work [5 3 2] -> can finish, releases its Allocation, Work = [7 4 3]
    P4: Need [4 3 1] <= Work [7 4 3] -> can finish, releases its Allocation, Work = [7 4 5]
  Pass 2:
    P0: Need [7 4 3] <= Work [7 4 5] -> can finish, releases its Allocation, Work = [7 5 5]
    P2: Need [6 0 0] <= Work [7 5 5] -> can finish, releases its Allocation, Work = [10 5 7]

RESULT: SAFE state. Safe sequence: <P1, P3, P4, P0, P2>
[PES1UG25CS001] ~/campuscompute$
```

S14: Safe state: the Need matrix is computed and a valid safe sequence is found.

S15 – Banker's unsafe state

```
RESULT: SAFE state. Safe sequence: <P1, P3, P4, P0, P2>
[PES1UG25CS001] ~/campuscompute$ ./campus banker tests/banker_unsafe.txt
=== Banker's safety check ===
State file : tests/banker_unsafe.txt
Processes  : 5, resource types: 3
Need = Maximum - Allocation

        Allocation   Maximum      Need
          A  B  C      A  B  C      A  B  C
  P0      0  1  0      7  5  3      7  4  3
  P1      2  0  0      3  2  2      1  2  2
  P2      3  0  2      9  0  2      6  0  0
  P3      2  1  1      2  2  2      0  1  1
  P4      0  0  2      4  3  3      4  3  1

  Available:  1  0  0   (resource types A B C)

Safety algorithm: Work = Available; scan P0..P4 in index order, repeat until no progress.
  Pass 1:
    P0: Need [7 4 3] > Work [1 0 0]  -> not enough, skip
    P1: Need [1 2 2] > Work [1 0 0]  -> not enough, skip
    P2: Need [6 0 0] > Work [1 0 0]  -> not enough, skip
    P3: Need [0 1 1] > Work [1 0 0]  -> not enough, skip
    P4: Need [4 3 1] > Work [1 0 0]  -> not enough, skip

RESULT: UNSAFE state. No safe sequence exists.
        Processes that can never be guaranteed to finish: P0, P1, P2, P3, P4 (possible deadlock)
[PES1UG25CS001] ~/campuscompute$
```

S15: Unsafe state correctly identified: no remaining process can complete with the available resources.

S16 – Normal worker process

```
[PES1UG25CS013] ~/campuscompute$ ./campus worker normal
[parent pid 12434] starting 3 workers, mode=normal
[worker 2 pid 12436] started
[worker 3 pid 12437] started
[worker 1 pid 12435] started
[worker 1 pid 12435] finished OK
[parent] worker 1 (pid 12435) exited normally (code 0) -> OK
[worker 2 pid 12436] finished OK
[parent] worker 2 (pid 12436) exited normally (code 0) -> OK
[worker 3 pid 12437] finished OK
[parent] worker 3 (pid 12437) exited normally (code 0) -> OK
Summary: 3 worker(s) OK, 0 failed -> detection PASS
```

S16: Child processes created with fork() and collected with waitpid(), all exiting normally.

S17 – Simulated worker failure

```
[PES1UG25CS013] ~/campuscompute$ ./campus worker fail
[parent pid 12525] starting 3 workers, mode=fail (failure of worker 2 is SIMULATED)
[worker 1 pid 12526] started
[worker 3 pid 12528] started
[worker 2 pid 12527] started
[worker 1 pid 12526] finished OK
[parent] worker 1 (pid 12526) exited normally (code 0) -> OK
[worker 2 pid 12527] *** SIMULATED FAILURE: exiting with code 42 (deliberate, not a real OS fault) ***
[parent] worker 2 (pid 12527) exited with NON-ZERO code 42 -> FAILURE DETECTED
[worker 3 pid 12528] finished OK
[parent] worker 3 (pid 12528) exited normally (code 0) -> OK
Summary: 2 worker(s) OK, 1 failed (simulated) -> detection PASS
```

S17: A worker fails on purpose (simulated, not a real OS fault) and the parent detects its non-zero exit status.

S18 – Resource-limit demonstration

```
[PES1UG25CS013] ~/campuscompute$ ./campus rlimit

== RLIMIT_NOFILE: open-file limit of 16 ==
  before: soft=1024 hard=524288
  after:  soft=16 hard=524288
  open("/dev/null") succeeded 13 time(s), then failed: Too many open files (errno 24)
  expected 13 successes = limit 16 - 3 descriptors already open
  [parent] child 12638 exited normally (code 0)
  Result: PASS - limit had the expected effect

== RLIMIT_AS: address-space limit of 256 MB ==
  before: soft=unlimited hard=unlimited
  after:  soft=268435456 hard=unlimited
  malloc(16 MB) succeeded 15 time(s) (240 MB), then returned NULL: Cannot allocate memory (errno 12)
  [parent] child 12639 exited normally (code 0)
  Result: PASS - limit had the expected effect

== RLIMIT_CPU: CPU-time limit of 1 second ==
  before: soft=unlimited hard=unlimited
  after:  soft=1 hard=2
  burning CPU in a busy loop...
  [parent] child 12640 was terminated by signal 24 (CPU time limit exceeded), core dumped
  Result: PASS - limit had the expected effect

rlimit summary: all demos PASS
```

S18: setrlimit() applied in a child process, with the observed effect of each limit.

S19 – Invalid input / command

```
berkman@berkman-VirtualBox
~/campuscompu

[PES1UG25CS001] ~/campuscompute$ ./campus foo;
campus: unknown command 'foo'
Run 'campus help' to see the available commands.
[PES1UG25CS001] ~/campuscompute$  ./campus sched rr tests/jobs_bad.txt
tests/jobs_bad.txt:3: CPU burst must be between 1 and 100000 (got '0')
tests/jobs_bad.txt:4: CPU burst 'abc' is not an integer
tests/jobs_bad.txt:5: expected 4 fields (id arrival burst priority) but found 3
tests/jobs_bad.txt:6: duplicate job ID 1 (first used by an earlier line)
tests/jobs_bad.txt:7: arrival time must be between 0 and 100000 (got '-1')
error: tests/jobs_bad.txt: 5 invalid line(s) - nothing was scheduled
[PES1UG25CS001] ~/campuscompute$
```

```
[PES1UG25CS013] ~/campuscompute$ ./campus worker bogus
worker: unknown mode 'bogus'
usage: campus worker normal|fail|signal
[PES1UG25CS013] ~/campuscompute$ ./campus logger 0
logger: invalid producer count '0'
usage: campus logger [producers 1-8] [consumers 1-4] [items 1-100]
```

S19: Invalid commands and malformed input produce clear error and usage messages.

S20 – Cleanup verification

```
[PES1UG25CS013] ~/campuscompute$ ./campus ipc > /dev/null; ./campus worker fail > /dev/null; ./campus logger > /dev/null
[PES1UG25CS013] ~/campuscompute$ echo "campus processes:"; ps -ef | grep '[c]ampus'
campus processes:
[PES1UG25CS013] ~/campuscompute$ echo "defunct processes:"; ps -ef | grep '[d]efunct'
defunct processes:
[PES1UG25CS013] ~/campuscompute$ echo "IPC objects:"; ls /dev/shm | grep campus
IPC objects:
```

S20: No leftover processes, zombies or IPC objects remain after the program runs.

S21 – Integrated execution

```
[PES1UG25CS013] ~/campuscompute$ ./campus all

============== [1/9] Scheduling comparison ==============

=== Scheduling comparison (same job set) ===
Input file : tests/jobs.txt (4 jobs)
Tie-break  : earlier arrival, then smaller job id
Quantum    : 2 (Round Robin)

Job    Arrival  Burst  Priority
J1           0      5         2
J2           1      3         1
J3           2      8         4
J4           3      6         3

Execution order:
  FCFS     : J1 -> J2 -> J3 -> J4
  RR       : J1 -> J2 -> J3 -> J1 -> J4 -> J2 -> J3 -> J1 -> J4 -> J3 -> J4 ->
             J3
  Priority : J1 -> J2 -> J4 -> J3

Policy      Avg CT  Avg TAT   Avg WT  Makespan Ctx switch
FCFS         12.75    11.25     5.75        22          3
RR           16.75    15.25     9.75        22         11
Priority     12.25    10.75     5.25        22          3

Lowest avg waiting time   : Priority (5.25)
Lowest avg turnaround time: Priority (10.75)
(CT = completion, TAT = turnaround, WT = waiting; Ctx switch = dispatches to a different job)

============== [2/9] Banker's: safe state ==============
=== Banker's safety check ===
State file : tests/banker_safe.txt
Processes  : 5, resource types: 3
Need = Maximum - Allocation

        Allocation   Maximum      Need
          A  B  C     A  B  C     A  B  C
  P0      0  1  0     7  5  3     7  4  3
  P1      2  0  0     3  2  2     1  2  2
  P2      3  0  2     9  0  2     6  0  0
  P3      2  1  1     2  2  2     0  1  1
  P4      0  0  2     4  3  3     4  3  1

  Available:  3  3  2   (resource types A B C)

Safety algorithm: Work = Available; scan P0..P4 in index order, repeat until no progress.
  Pass 1:
    P0: Need [7 4 3] > Work [3 3 2]  -> not enough, skip
    P1: Need [1 2 2] <= Work [3 3 2] -> can finish, releases its Allocation, Work = [5 3 2]
    P2: Need [6 0 0] > Work [5 3 2]  -> not enough, skip
    P3: Need [0 1 1] <= Work [5 3 2] -> can finish, releases its Allocation, Work = [7 4 3]
    P4: Need [4 3 1] <= Work [7 4 3] -> can finish, releases its Allocation, Work = [7 4 5]
  Pass 2:
    P0: Need [7 4 3] <= Work [7 4 5] -> can finish, releases its Allocation, Work = [7 5 5]
    P2: Need [6 0 0] <= Work [7 5 5] -> can finish, releases its Allocation, Work = [10 5 7]

RESULT: SAFE state. Safe sequence: <P1, P3, P4, P0, P2>
```

```
=============== [3/9] Banker's: unsafe state ===============
=== Banker's safety check ===
State file : tests/banker_unsafe.txt
Processes  : 5, resource types: 3
Need = Maximum - Allocation

       Allocation   Maximum      Need
         A  B  C      A  B  C      A  B  C
  P0     0  1  0      7  5  3      7  4  3
  P1     2  0  0      3  2  2      1  2  2
  P2     3  0  2      9  0  2      6  0  0
  P3     2  1  1      2  2  2      0  1  1
  P4     0  0  2      4  3  3      4  3  1

  Available:  1  0  0   (resource types A B C)

Safety algorithm: Work = Available; scan P0..P4 in index order, repeat until no progress.
  Pass 1:
    P0: Need [7 4 3] > Work [1 0 0]  -> not enough, skip
    P1: Need [1 2 2] > Work [1 0 0]  -> not enough, skip
    P2: Need [6 0 0] > Work [1 0 0]  -> not enough, skip
    P3: Need [0 1 1] > Work [1 0 0]  -> not enough, skip
    P4: Need [4 3 1] > Work [1 0 0]  -> not enough, skip

RESULT: UNSAFE state. No safe sequence exists.
        Processes that can never be guaranteed to finish: P0, P1, P2, P3, P4 (possible deadlock)

=============== [4/9] Producer-consumer logger ===============
Logger: 3 producer(s), 1 consumer(s), 5 item(s) each, queue capacity 5
[consumer-1] queue EMPTY -> waiting
[producer-2] produced seq=1 (queue 1/5)
[producer-2] produced seq=2 (queue 2/5)
[producer-2] produced seq=3 (queue 3/5)
[producer-2] produced seq=4 (queue 4/5)
[producer-2] produced seq=5 (queue 5/5)
[consumer-1] consumed seq=1 (queue 4/5)
[producer-3] produced seq=6 (queue 5/5)
[producer-3] queue FULL (5/5) -> waiting
[producer-1] queue FULL (5/5) -> waiting
[consumer-1] consumed seq=2 (queue 4/5)
[producer-3] produced seq=7 (queue 5/5)
[producer-3] queue FULL (5/5) -> waiting
[consumer-1] consumed seq=3 (queue 4/5)
[producer-1] produced seq=8 (queue 5/5)
[producer-1] queue FULL (5/5) -> waiting
[consumer-1] consumed seq=4 (queue 4/5)
[producer-3] produced seq=9 (queue 5/5)
[producer-3] queue FULL (5/5) -> waiting
[consumer-1] consumed seq=5 (queue 4/5)
[producer-1] produced seq=10 (queue 5/5)
[producer-1] queue FULL (5/5) -> waiting
[consumer-1] consumed seq=6 (queue 4/5)
[producer-3] produced seq=11 (queue 5/5)
[producer-3] queue FULL (5/5) -> waiting
[consumer-1] consumed seq=7 (queue 4/5)
[producer-1] produced seq=12 (queue 5/5)
[producer-1] queue FULL (5/5) -> waiting
[consumer-1] consumed seq=8 (queue 4/5)
[producer-3] produced seq=13 (queue 5/5)
[consumer-1] consumed seq=9 (queue 4/5)
[producer-1] produced seq=14 (queue 5/5)
[producer-1] queue FULL (5/5) -> waiting
[consumer-1] consumed seq=10 (queue 4/5)
[producer-1] produced seq=15 (queue 5/5)
All producers joined.
[consumer-1] consumed seq=11 (queue 4/5)
[consumer-1] consumed seq=12 (queue 3/5)
[consumer-1] consumed seq=13 (queue 2/5)
[consumer-1] consumed seq=14 (queue 1/5)
[consumer-1] consumed seq=15 (queue 0/5)
[consumer-1] finished, wrote 15 entries
All consumers joined.
Summary: expected=15 produced=15 consumed=15 log_lines=15  -> PASS
Log file: logs/campus.log
```

```
=============== [6/9] Worker: normal completion ===============
[parent pid 20355] starting 3 workers, mode=normal
[worker 1 pid 20365] started
[worker 3 pid 20367] started
[worker 2 pid 20366] started
[worker 1 pid 20365] finished OK
[parent] worker 1 (pid 20365) exited normally (code 0) -> OK
[worker 2 pid 20366] finished OK
[parent] worker 2 (pid 20366) exited normally (code 0) -> OK
[worker 3 pid 20367] finished OK
[parent] worker 3 (pid 20367) exited normally (code 0) -> OK
Summary: 3 worker(s) OK, 0 failed -> detection PASS

=============== [7/9] Worker: simulated failure ===============
[parent pid 20355] starting 3 workers, mode=fail (failure of worker 2 is SIMULATED)
[worker 1 pid 20368] started
[worker 2 pid 20369] started
[worker 3 pid 20370] started
[worker 1 pid 20368] finished OK
[parent] worker 1 (pid 20368) exited normally (code 0) -> OK
[worker 2 pid 20369] *** SIMULATED FAILURE: exiting with code 42 (deliberate, not a real OS fault) ***
[parent] worker 2 (pid 20369) exited with NON-ZERO code 42 -> FAILURE DETECTED
[worker 3 pid 20370] finished OK
[parent] worker 3 (pid 20370) exited normally (code 0) -> OK
Summary: 2 worker(s) OK, 1 failed (simulated) -> detection PASS

=============== [8/9] Resource limits ===============

== RLIMIT_NOFILE: open-file limit of 16 ==
  before: soft=1024 hard=524288
  after:  soft=16 hard=524288
  open("/dev/null") succeeded 13 time(s), then failed: Too many open files (errno 24)
  expected 13 successes = limit 16 - 3 descriptors already open
  [parent] child 20371 exited normally (code 0)
  Result: PASS - limit had the expected effect

== RLIMIT_AS: address-space limit of 256 MB ==
  before: soft=unlimited hard=unlimited
  after:  soft=268435456 hard=unlimited
  malloc(16 MB) succeeded 12 time(s) (192 MB), then returned NULL: Cannot allocate memory (errno 12)
  [parent] child 20372 exited normally (code 0)
  Result: PASS - limit had the expected effect

== RLIMIT_CPU: CPU-time limit of 1 second ==
  before: soft=unlimited hard=unlimited
  after:  soft=1 hard=2
  burning CPU in a busy loop...
  [parent] child 20373 was terminated by signal 24 (CPU time limit exceeded), core dumped
  Result: PASS - limit had the expected effect

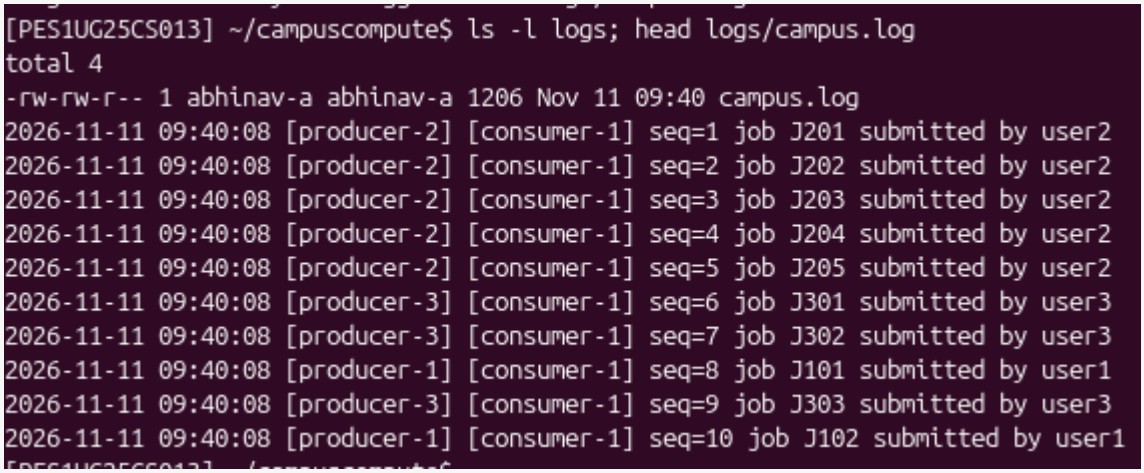
rlimit summary: all demos PASS

=============== [9/9] Cleanup verification ===============
  no child processes remain
  threads in this process: 1 (expected 1)
  our shared-memory/semaphore objects left: 0 (expected 0)

=============== INTEGRATED SUMMARY ===============
 PASS Scheduling comparison           return code 0 (expected 0)
 PASS Banker's: safe state            return code 0 (expected 0)
 PASS Banker's: unsafe state          return code 1 (expected 1)
 PASS Producer-consumer logger        return code 0 (expected 0)
 PASS Shared memory + semaphores      return code 0 (expected 0)
 PASS Worker: normal completion       return code 0 (expected 0)
 PASS Worker: simulated failure       return code 0 (expected 0)
 PASS Resource limits                 return code 0 (expected 0)
 PASS No leftover child processes     no child processes remain
 PASS All worker threads ended        threads in this process: 1 (expected 1)
 PASS IPC objects released            our shared-memory/semaphore objects left: 0 (expected 0)
-------------------------------------------------
 11 of 11 checks passed  -> ALL OK
 Log file written by the logger demo: logs/campus.log
```

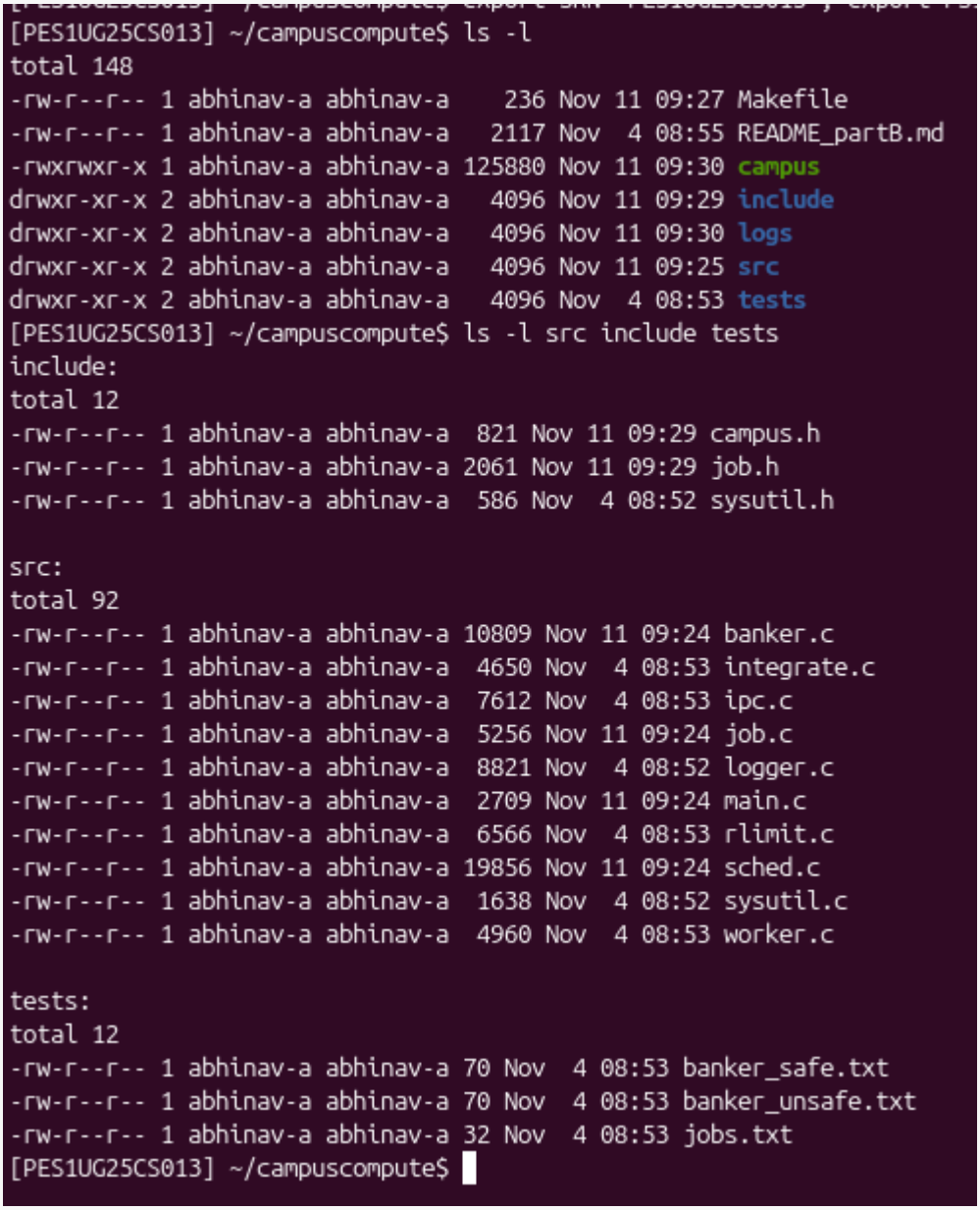
S21: Integrated demonstration running every feature, ending with the summary table.

S22 – Generated logs



S22: The log file created by the program and a sample of its contents.

S23 – Final source and build files



S23: Final source files, the Makefile and the built executable.

S24 – Submission ZIP

```
[PES1UG25CS013] ~/campuscompute$ unzip -l PES1UG25CS013.zip
Archive:  PES1UG25CS013.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
      236  2026-11-11 09:27   Makefile
        0  2026-11-11 09:29   include/
      821  2026-11-11 09:29   include/campus.h
     2061  2026-11-11 09:29   include/job.h
      586  2026-11-04 08:52   include/sysutil.h
        0  2026-11-11 09:25   src/
     7612  2026-11-04 08:53   src/ipc.c
     5256  2026-11-11 09:24   src/job.c
     2709  2026-11-11 09:24   src/main.c
     4960  2026-11-04 08:53   src/worker.c
     4650  2026-11-04 08:53   src/integrate.c
    19856  2026-11-11 09:24   src/sched.c
    10809  2026-11-11 09:24   src/banker.c
     1638  2026-11-04 08:52   src/sysutil.c
     8821  2026-11-04 08:52   src/logger.c
     6566  2026-11-04 08:53   src/rlimit.c
        0  2026-11-04 08:53   tests/
       32  2026-11-04 08:53   tests/jobs.txt
       70  2026-11-04 08:53   tests/banker_safe.txt
       70  2026-11-04 08:53   tests/banker_unsafe.txt
---------                     -------
    76753                     20 files
[PES1UG25CS013] ~/campuscompute$   unzip -t PES1UG25CS013.zip
Archive:  PES1UG25CS013.zip
    testing: Makefile                   OK
    testing: include/                   OK
    testing: include/campus.h           OK
    testing: include/job.h              OK
    testing: include/sysutil.h          OK
    testing: src/                       OK
    testing: src/ipc.c                  OK
    testing: src/job.c                  OK
    testing: src/main.c                 OK
    testing: src/worker.c               OK
    testing: src/integrate.c            OK
    testing: src/sched.c                OK
    testing: src/banker.c               OK
    testing: src/sysutil.c              OK
    testing: src/logger.c               OK
    testing: src/rlimit.c               OK
    testing: tests/                     OK
    testing: tests/jobs.txt             OK
    testing: tests/banker_safe.txt      OK
    testing: tests/banker_unsafe.txt    OK
No errors detected in compressed data of PES1UG25CS013.zip.
```

S24: Contents of the submission archive and a successful integrity test.